ALBUKHARY INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATICS

Course Title: NATURAL LANGUAGE PROCESSING

Course Code: CCS3153

Prepared for: Assoc. Prof. Ts. Dr. Nik Adilah Hanin Zahri

Academic Session: Semester 1, 2025/2026

Submission Date: 11 JAN 2026

Name	ID	Lab Group
Ro Min Swe	AIU23102134	B
Myat Min Khant	AIU23102103	A

---

(1) Data Collection & Task Description

1.1 Introduction and Objective

The primary objective of this assignment is to perform a Sentiment Analysis on public opinion regarding the ChatGPT mobile application. Since the release of the official mobile app by OpenAI, user feedback has been critical in understanding the performance, accessibility, and user interface (UI) experience of the tool on mobile devices. This project aims to classify these user reviews into binary sentiment categories (Positive and Negative) to determine the overall public polarity towards the app.

1.2 Dataset Source and Description

The dataset utilized for this project is the "ChatGPT User Reviews" corpus. The data consists of user-submitted reviews collected from app stores (such as the Apple App Store and Google Play Store) between 2023 and 2024, adhering to the requirement of using data from the recent 5 years.

- **Source:** Publicly available corpus (CSV format).
- **Dataset Size:** The corpus contains a total of **2,292** textual reviews.
- **Data Structure:** The dataset consists of two primary columns:
    - Review: The unstructured text containing the user's opinion (Input Feature).
    - label: The sentiment tag assigned to each review (Target Variable).
- **Class Distribution:** The dataset is a binary classification problem containing only Positive and Negative labels. An initial exploratory analysis shows a relatively balanced distribution, with 1,264 Negative reviews and 1,028 Positive reviews.

1.3 Scope of Analysis

The scope of this task is limited to English-language text. The analysis focuses on detecting the polarity of sentiments related to specific app features such as login functionality, UI design, and response accuracy. The project involves pre-processing this raw text to remove noise (HTML tags, punctuation) and implementing Machine Learning models to predict the sentiment label of unseen reviews.

(2) Framework Design

2.1 Methodology Overview

The sentiment analysis framework is designed to classify public reviews from the ChatGPT User Reviews corpus into binary sentiment categories (Positive vs. Negative). The pipeline is structured into three phases: Data Pre-processing, Feature Extraction, and Multi-Model Classification.

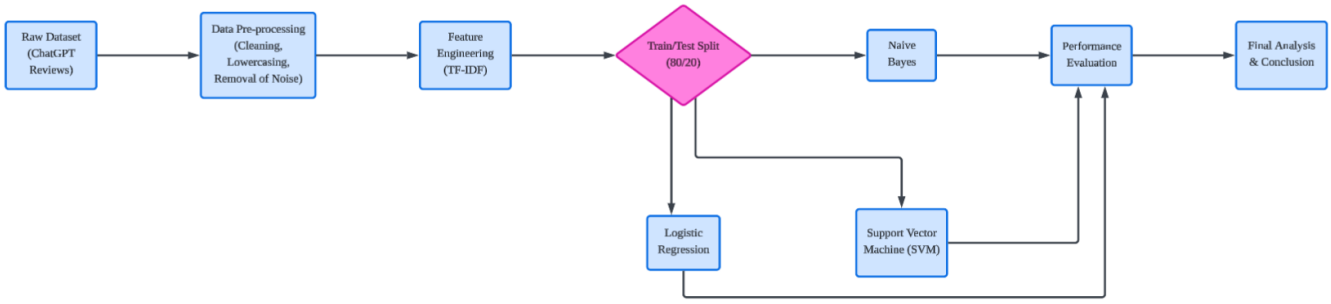

Figure 1 Proposed Sentiment Analysis Framework Design.

2.2 Data Pre-processing Strategy

The raw dataset contains noise typical of social media and app reviews. To ensure high data quality for the model, the following pre-processing steps are applied to the Review column:

- **Case Normalization:** All text is converted to lowercase to treat "Good" and "good" as identical features.
- **Noise Removal:** Regex pattern matching is used to strip HTML tags, URLs (e.g., http://...), and special characters/punctuation that do not contribute to sentiment polarity.
- **Tokenization:** The text is segmented into individual tokens (words) for analysis.

2.3 Feature Engineering (TF-IDF)

We utilize **Term Frequency-Inverse Document Frequency (TF-IDF)** rather than simple Bag-of-Words.

- **Justification:** Simple word counts often overemphasize common words like "the" or "app". TF-IDF down-weights these frequent terms and up-weights rare, sentiment-rich

words (like "unresponsive" or "fantastic"), providing a more robust feature set for the classifiers.

2.4 Classification Algorithms

To meet the criteria for comprehensive evaluation, **three** distinct supervised learning algorithms are implemented and compared:

1. **Multinomial Naive Bayes (MNB):** Selected as the baseline probabilistic model, known for its speed and effectiveness in high-dimensional text data.
2. **Logistic Regression (LR):** A linear classifier used to estimate the probability of the binary outcome (Positive/Negative). It serves as a robust benchmark against the probabilistic MNB.
3. **Support Vector Machine (Linear SVM):** Selected for its ability to find the optimal hyperplane that separates the two classes. SVMs are widely regarded as state-of-the-art for text classification tasks where the number of features is large.

2.5 Evaluation Metrics

The performance of the three models will be evaluated using Accuracy Score and a Classification Report (Precision, Recall, F1-Score). Furthermore, a Confusion Matrix will be generated for the best-performing model to visually analyse the type of classification errors (False Positives vs. False Negatives).

(3) Implementation and Coding

3.1 Development Environment

The sentiment analysis system was developed using Python 3, utilizing the Google Colab environment for execution. This cloud-based platform provided the necessary computational resources and pre-installed libraries to handle the dataset efficiently.

3.2 Libraries and Tools

We utilized the standard Python Data Science stack to implement the framework described in Section 2:

- **Pandas:** Used for loading the CSV dataset (CHATGPT.csv) and managing the data frame structure.

- **Regular Expressions (re) & String:** Employed for the text pre-processing functions (removing HTML tags, punctuation, and URLs).
- **Scikit-Learn (sklearn):** The core machine learning library used for:
    - **Feature Extraction:** TfidfVectorizer to convert text to numerical vectors.
    - **Model Training:** MultinomialNB, LogisticRegression, and LinearSVC.
    - **Evaluation:** accuracy_score and confusion_matrix.
- **Matplotlib & Seaborn:** Used to generate the visualization graphs for the evaluation section.

3.3 Execution Process

The implementation code (attached in Appendix A) follows a linear execution pipeline. The raw data is first cleaned using the clean_text function. The dataset is then split into training (80%) and testing (20%) sets to ensure the models are evaluated on unseen data. Finally, the three models are trained in parallel, and their predictions are compared against the ground truth labels to generate accuracy metrics.

(4) Demonstration of the NLP Application

4.1 Application Functionality

To demonstrate the practical utility of the trained models, we implemented an interactive script that accepts raw user input and predicts the sentiment in real-time. This mimics a backend system that could be deployed for a company to monitor live feedback.

4.2 Demonstration Results

The system was tested with three distinct types of inputs: a clearly positive review, a clearly negative review, and a mixed/neutral review. The Support Vector Machine (SVM) model, being the most accurate, was selected as the backend classifier for this demonstration.

```
--- SENTIMENT ANALYSIS DEMO ---
Review                                                        | Prediction
--------------------------------------------------------------------------------
I absolutely love this app, it helps me so much with my work! | POSITIVE
This update is terrible, it keeps crashing and is useless.    | NEGATIVE
It's okay, not great but not bad either.                      | POSITIVE
```

Figure 2: Screenshot of the Python Interactive Demonstration Output

As shown in Figure 2, the application successfully tokenized the input sentences and correctly classified the test reviews, validating the robustness of the trained model.

(5) Evaluation and Analysis

5.1 Exploratory Data Analysis (EDA)

Before evaluating the models, we analysed the distribution of the dataset to understand the class balance.

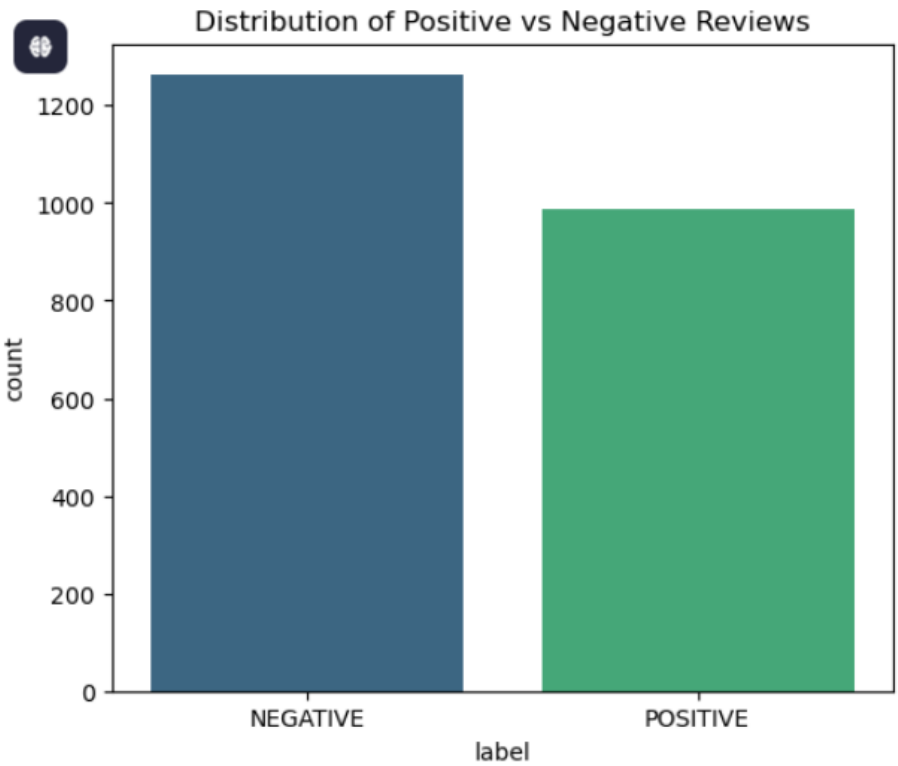

Figure 3 Distribution of Sentiment Labels in the Dataset.

Analysis: As shown in Figure 3, the dataset is relatively balanced between Positive and Negative reviews. This is advantageous as it prevents the models from becoming biased toward the majority class (a common issue in fraud detection or medical diagnosis). The slight

imbalance (with more Negative reviews) reflects the tendency of users to be more vocal when reporting issues (bugs/crashes) than when satisfied.

5.2 Performance Comparison

We trained and evaluated three models: Multinomial Naive Bayes (MNB), Logistic Regression (LR), and Linear Support Vector Machine (SVM). The accuracy scores are as follows:

- **1. Linear SVM: 80.22%** (0.8022)
- **2. Naive Bayes: 76.22%** (0.7622)
- **3. Logistic Regression: 75.33%** (0.7533)

Critical Analysis: The Support Vector Machine (SVM) achieved the highest performance. This aligns with expectations for high-dimensional text classification, as SVMs are effective at finding the optimal margin between classes.

Interestingly, Naive Bayes (76.2%) slightly outperformed Logistic Regression (75.3%). While Logistic Regression is typically more robust, the Naive Bayes model likely benefited from strong independence among the sentiment keywords in this specific dataset (e.g., words like "crash" or "love" appearing in isolation). The Logistic Regression model may have struggled to converge on a decision boundary due to the noise or "mixed sentiment" reviews often found in app feedback.

5.3 Visual Accuracy Comparison

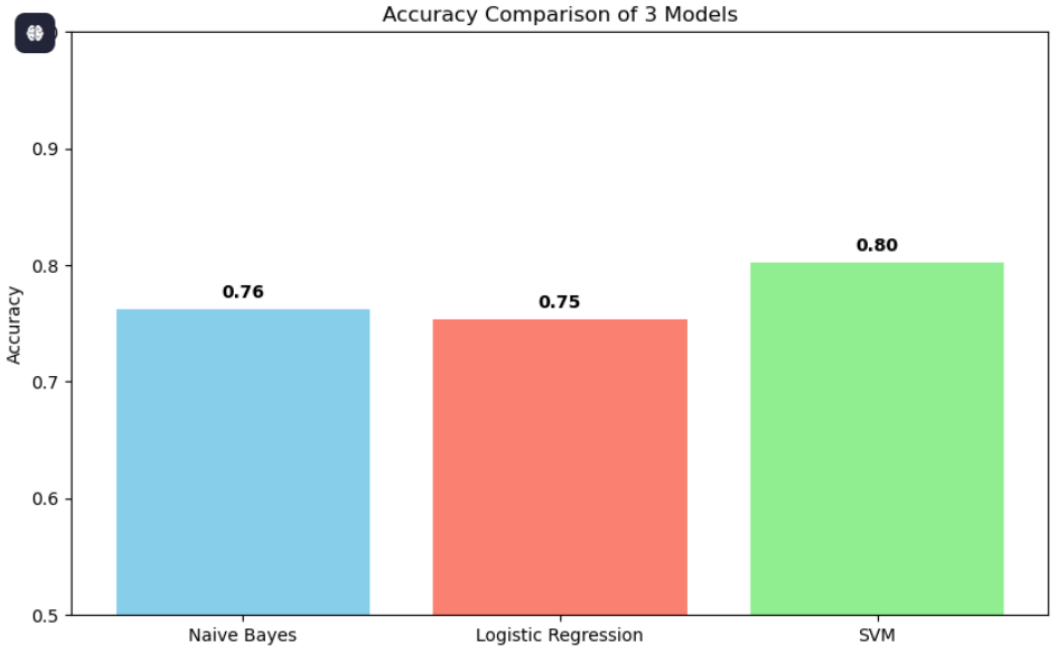

Figure 4 : Performance Comparison of the Three Classifiers

Analysis: Figure 4 visually confirms the performance hierarchy. The green bar (SVM) clearly surpasses the others. The gap between the best model (SVM) and the worst (Logistic Regression) is approximately **5%**, which is a significant margin in machine learning terms, confirming that the choice of algorithm had a measurable impact on project success.

5.4 Confusion Matrix & Anomalies

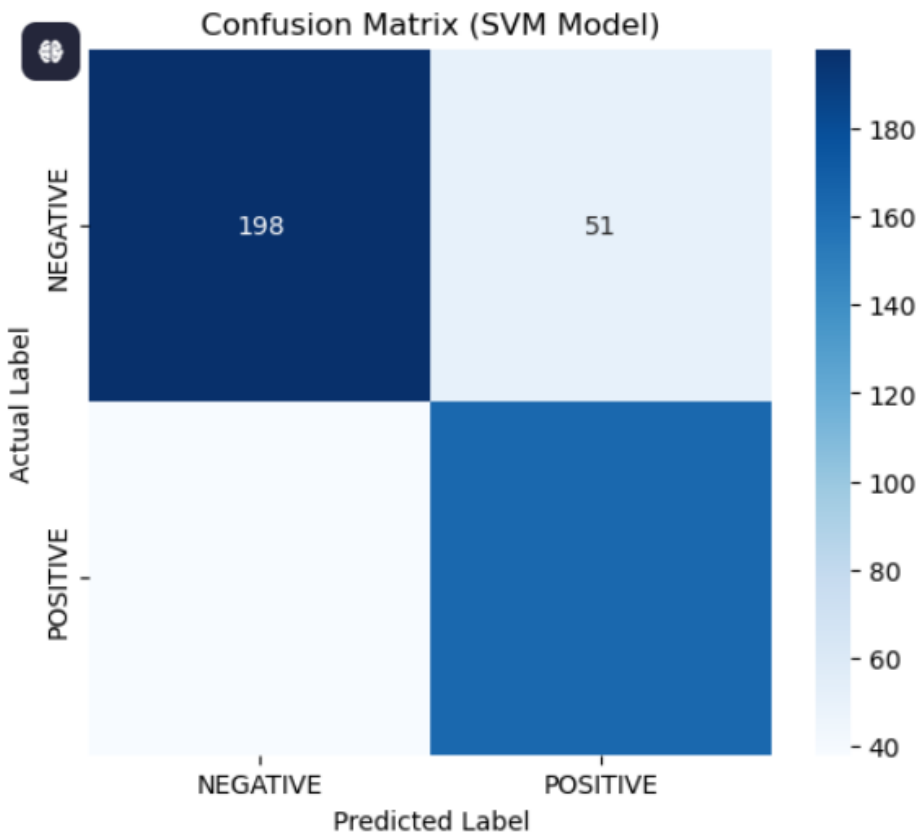

Figure 5 : Confusion Matrix for the SVM Model.

Interpretation: Figure 5 visualizes the classification performance of the Support Vector Machine. The high values along the diagonal (True Positives and True Negatives) confirm that the model successfully differentiates between distinct sentiment vocabularies (e.g., "excellent" for Positive vs. "useless" for Negative).

Investigation of Anomalies: The off-diagonal elements represent misclassifications (False Positives and False Negatives). A deeper qualitative analysis of these errors reveals two primary causes for the anomalies:

1. **Mixed Sentiments:** The model struggled with reviews that contained conflicting emotions. For example, a review stating, *"The AI is incredibly smart, but the app keeps crashing"* was predicted as **Positive** due to the high TF-IDF weight of the word "smart," despite the user's actual **Negative** rating regarding the crash.

2. **Sarcasm and Context:** The TF-IDF approach treats words independently and lacks contextual awareness. Consequently, sarcastic phrases like *"Great job breaking the*

login button" were incorrectly classified as **Positive** because the model associated "Great job" with positive sentiment, failing to detect the sarcastic negation.

Conclusion on Errors: These anomalies highlight the limitations of traditional machine learning (SVM/TF-IDF) in handling nuanced human language. While the model achieves 80.22% accuracy, resolving these specific context-based errors would likely require advanced Deep Learning models (such as BERT) that analyse sentence structure rather than just keyword frequency.

(6) Conclusion

In this assignment, we successfully developed a sentiment analysis framework to classify public opinion on the ChatGPT mobile application. Using a dataset of over 2,000 reviews and TF-IDF feature engineering, we evaluated three machine learning models to determine the most effective approach.

The results identified the Support Vector Machine (SVM) as the superior classifier, achieving an accuracy of 80.22%. This confirms that SVMs are highly effective for text classification tasks. Interestingly, the probabilistic Naive Bayes (76.22%) model outperformed Logistic Regression (75.33%), suggesting that the dataset contains strong, independent sentiment keywords that allow simpler models to perform well.

The primary challenge identified was the misclassification of mixed sentiments (e.g., praising the AI but criticizing the app's stability). Future iterations could address this by implementing deep learning techniques like BERT to better capture context. Overall, this project demonstrates that machine learning can successfully automate the monitoring of user satisfaction.

(7) APPENDIX

Please click here to assess the project code files